

Bidirectional Encoder Representations from Transformers



Bidirectional Encoder Representations from Transformers

Like *Bidirectional LSTMs*, let's look in **both** directions



Bidirectional Encoder Representations from Transformers

Let's only use Transformer *Encoders*, no Decoders



Bidirectional Encoder Representations from Transformers

It's a language model that builds rich representations



A man with a beard and a blue cap, wearing a colorful patterned shirt, is on the left. Elmo is on the right. They are in a room with a blue wall and a wooden bench. Four speech bubbles are overlaid on the image, containing a conversation about word embeddings.

Hey ELMo, what's the embedding
of the word "stick"?

There are multiple possible
embeddings! Use it in a sentence.

Oh, okay. Here:
"Let's stick to improvisation in this
skit"

Oh in that case, the embedding is:
-0.02, -0.16, 0.12, -0.1etc

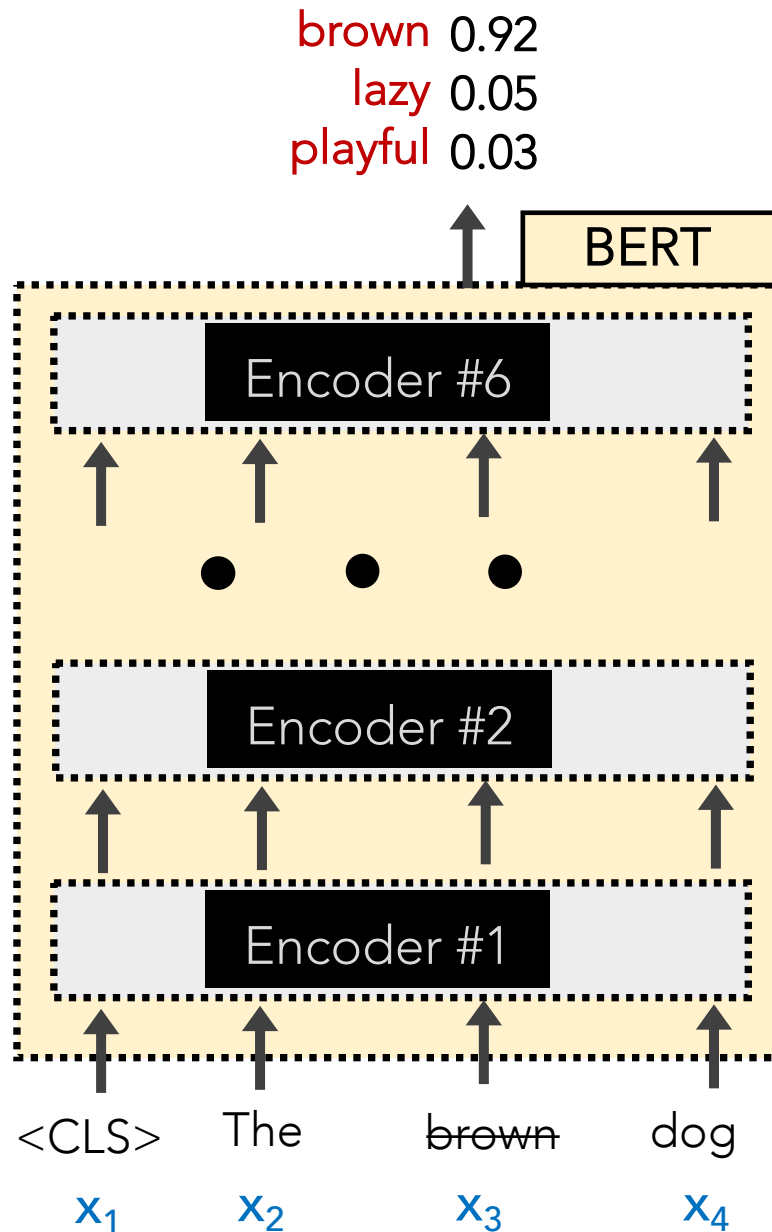
Naming convention

Many deep learning models, including pre-trained ones with cute names (e.g., ELMo, BERT, ALBERT, GPT-3), refer to an exact combination of:

- The **model's architecture**
- The **training objective** to pre-train (e.g., MLM prediction)
- The **data** (e.g., Google BooksCorpus, Wikipedia)

Many people abuse the terms and swap out components.

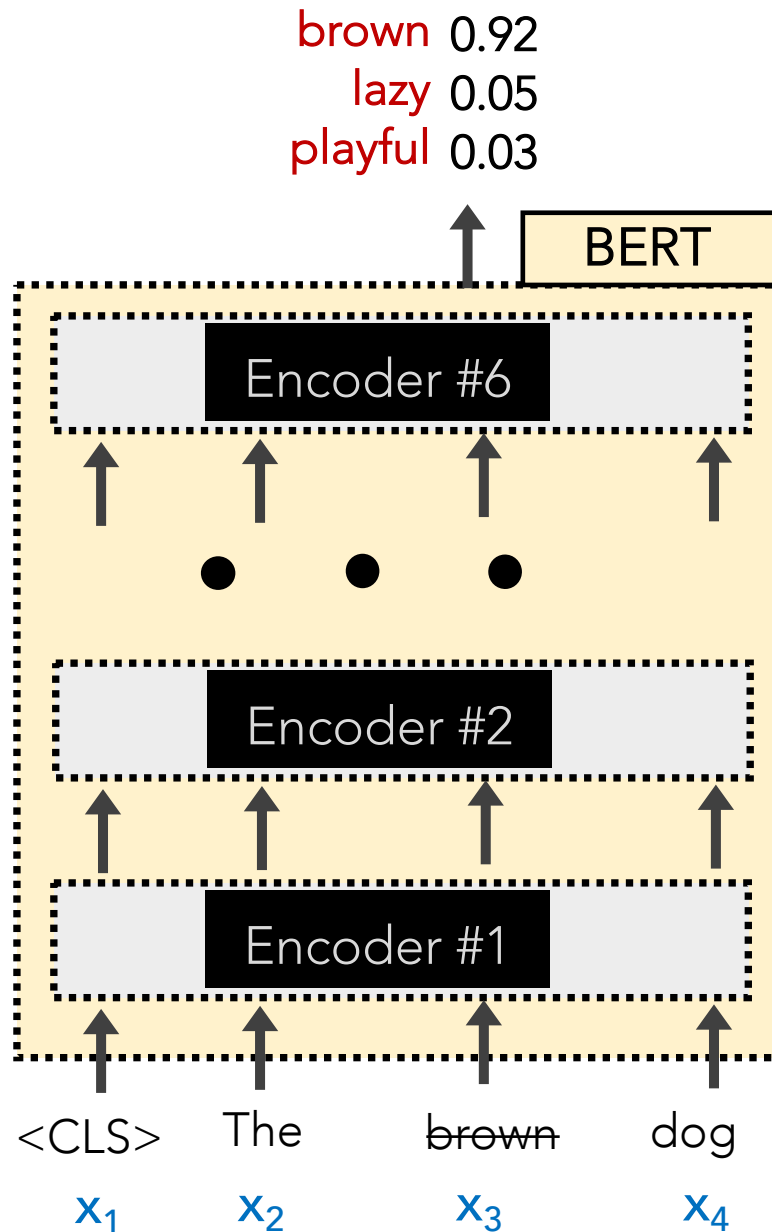
BERT



BERT:

- **Model:** several Transformer Encoders. Input sentence or sentence pairs, [CLS] token, subword embeddings
- **Objective:** MLM and next-sentence prediction
- **Data:** BooksCorpus and Wikipedia

BERT



BERT has 2 training objectives:

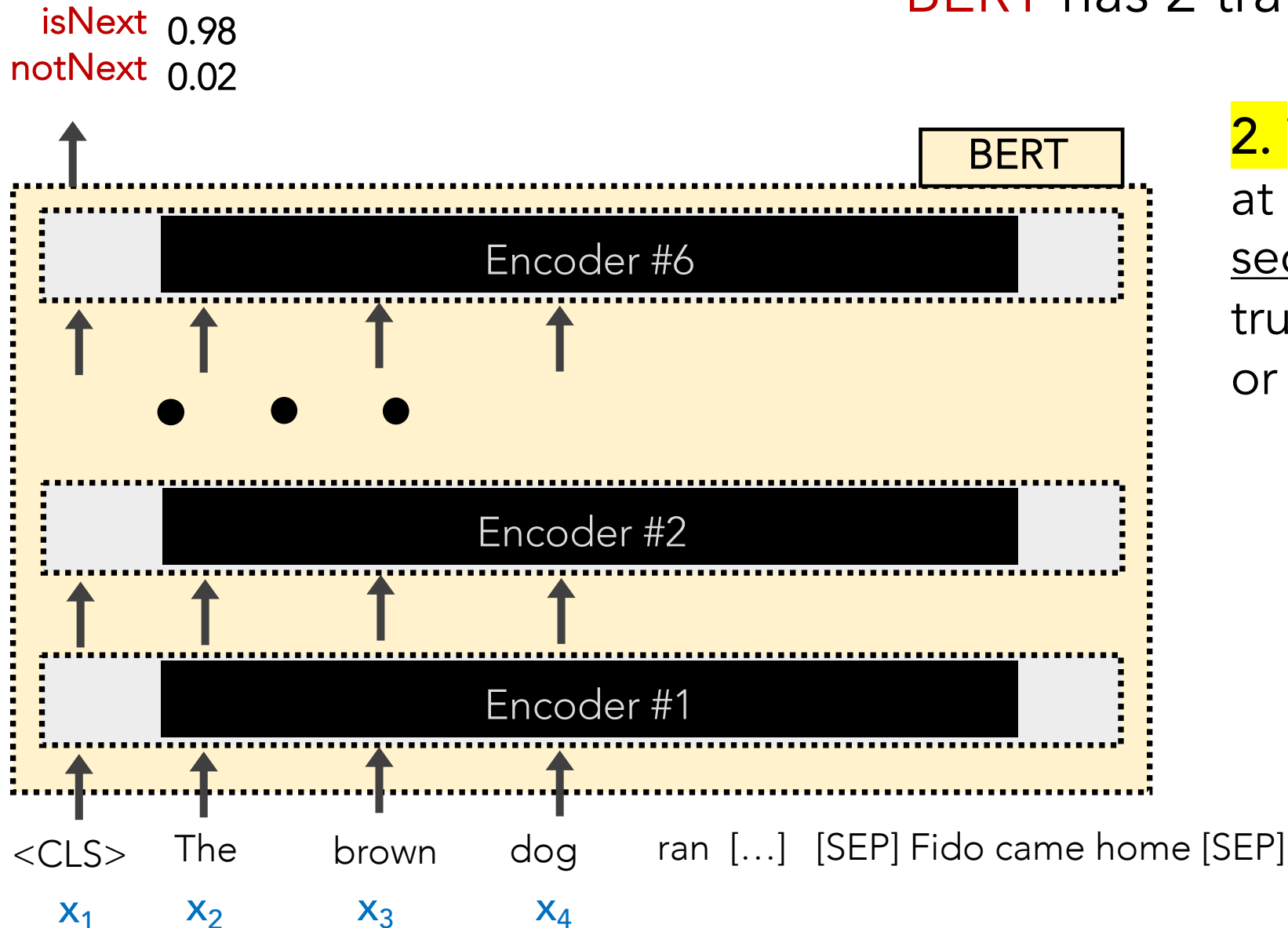
1. Predict the **Masked word** (a la CBOW)

15% of all input words are randomly masked.

- 80% become [MASK]
- 10% become revert back
- 10% become are deliberately corrupted as wrong words

BERT

BERT has 2 training objectives:

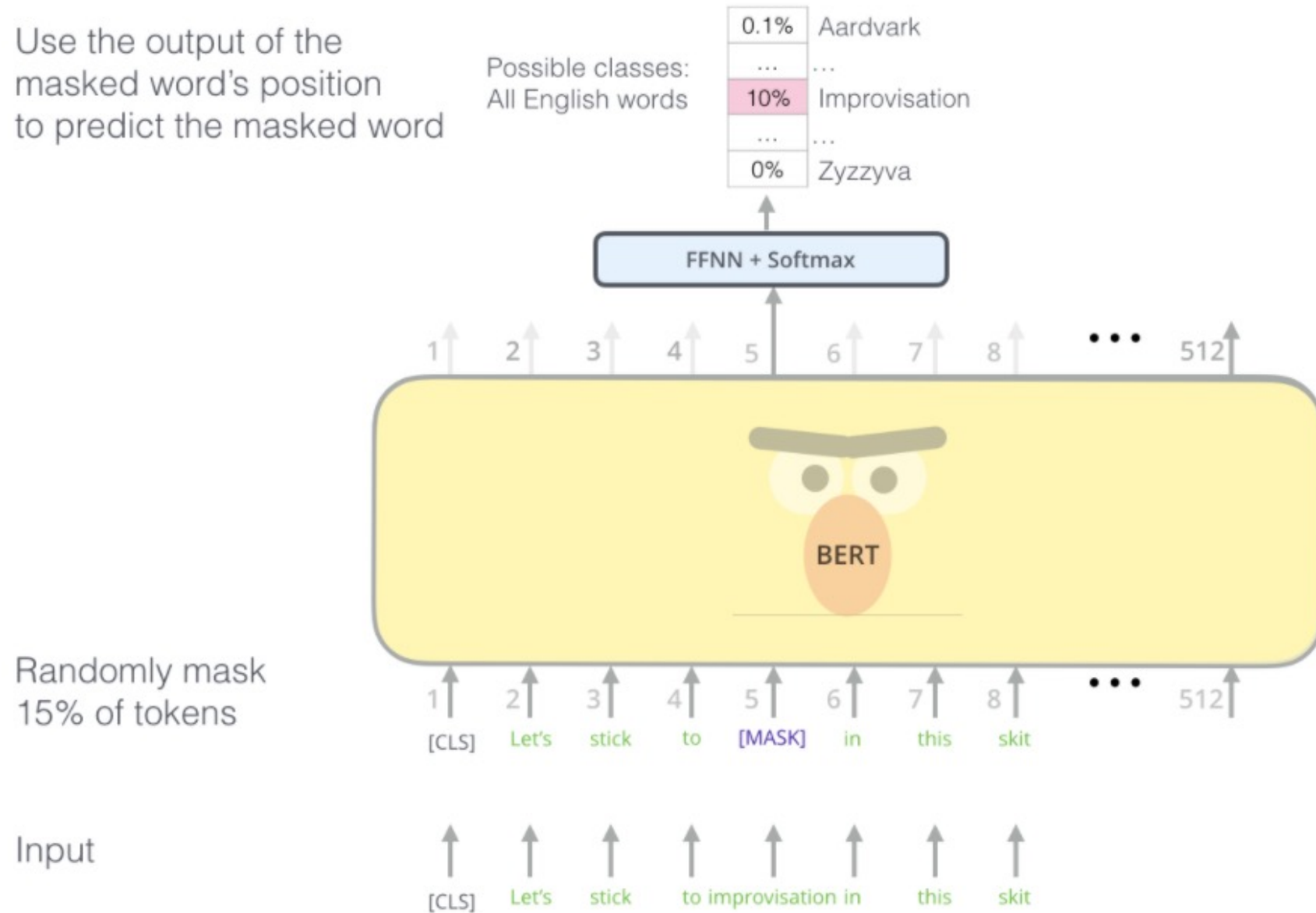


2. Two sentences are fed in at a time. Predict the if the second sentence of input truly follows the first one or not.

BERT (alternate view)

<https://jalammar.github.io/illustrated-bert/>

Use the output of the masked word's position to predict the masked word

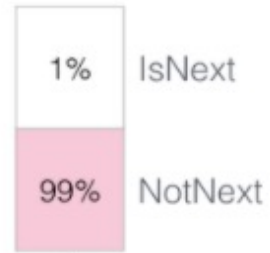


BERT's clever language modeling task masks 15% of words in the input and asks the model to predict the missing word.

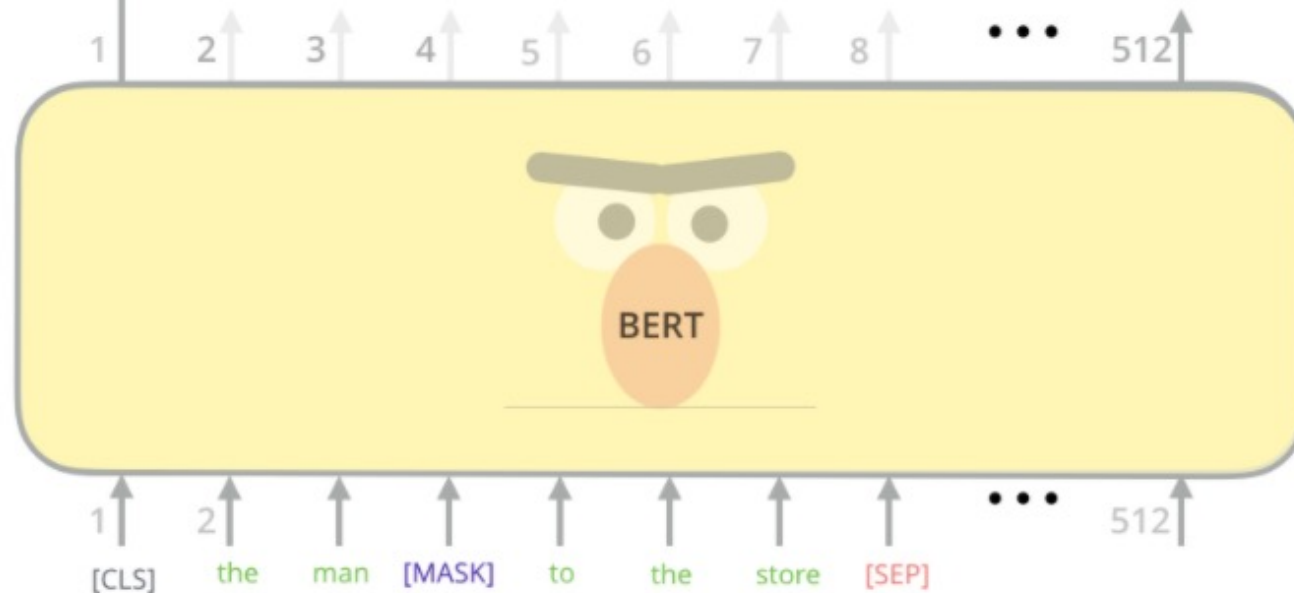
BERT (alternate view)

<https://jalammar.github.io/illustrated-bert/>

Predict likelihood
that sentence B
belongs after
sentence A



FFNN + Softmax



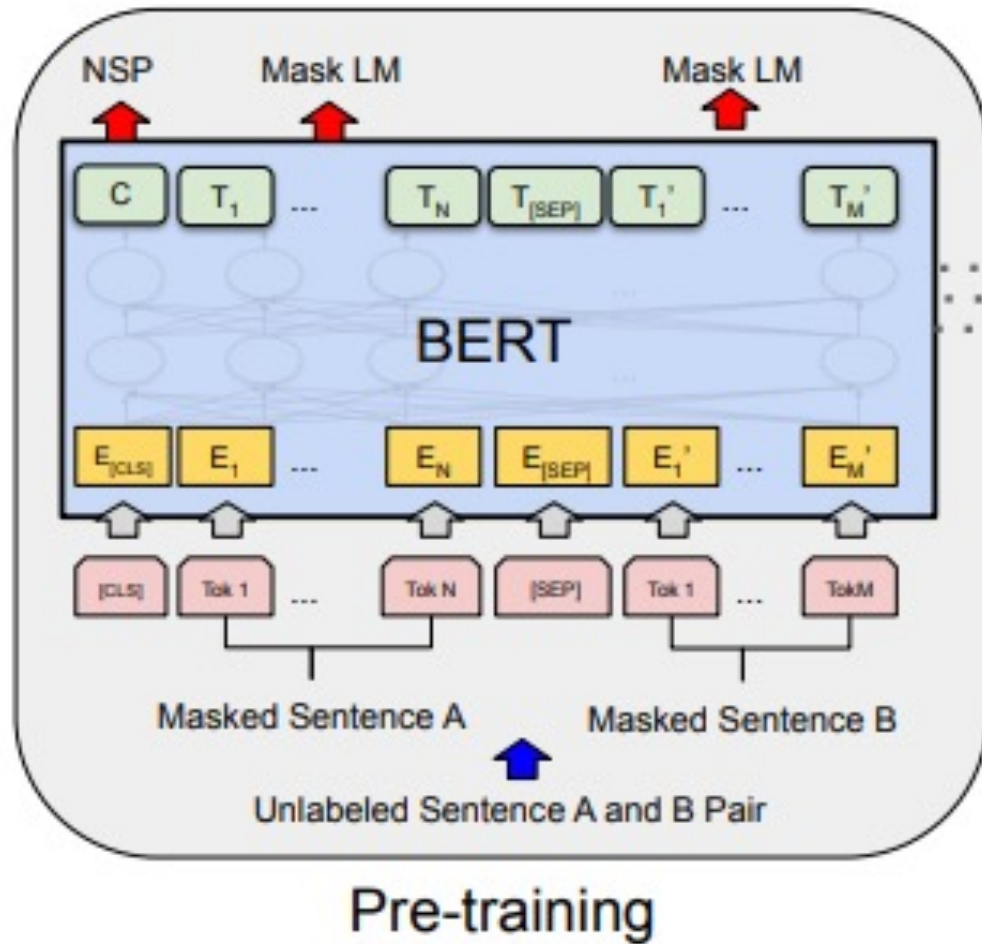
Tokenized
Input

Input

[CLS] the man [MASK] to the store [SEP] penguin [MASK] are flightless birds [SEP]

Sentence A Sentence B

BERT

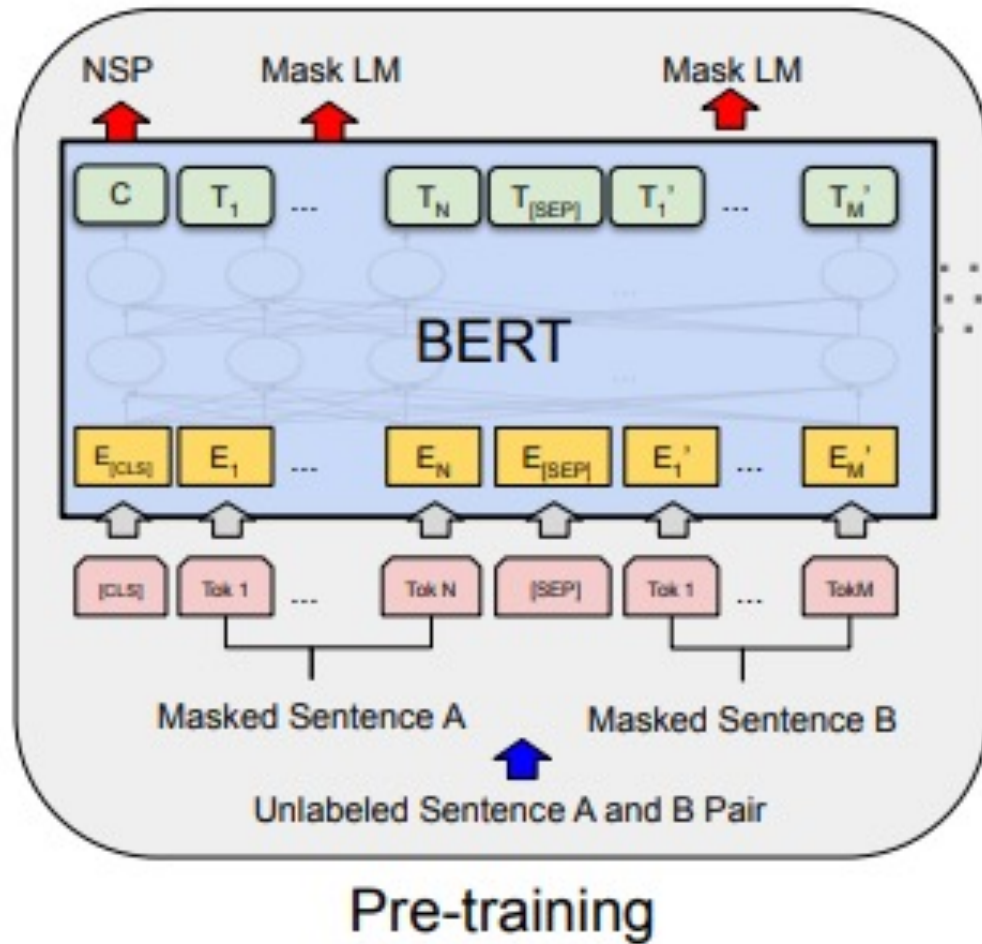


The two sentences are separated by a **<SEP>** token.

50% of the time, the 2nd sentence is a **randomly selected sentence** from the corpus.

50% of the time, it **truly follows the first sentence** in the corpus.

BERT



NOTE: BERT also embeds the inputs by their **WordPiece** embeddings.

WordPiece is a sub-word tokenization learns to merge and use characters based on which pairs maximize the likelihood of the training data if added to the vocab.

BERT's inputs

<https://arxiv.org/pdf/1810.04805.pdf>

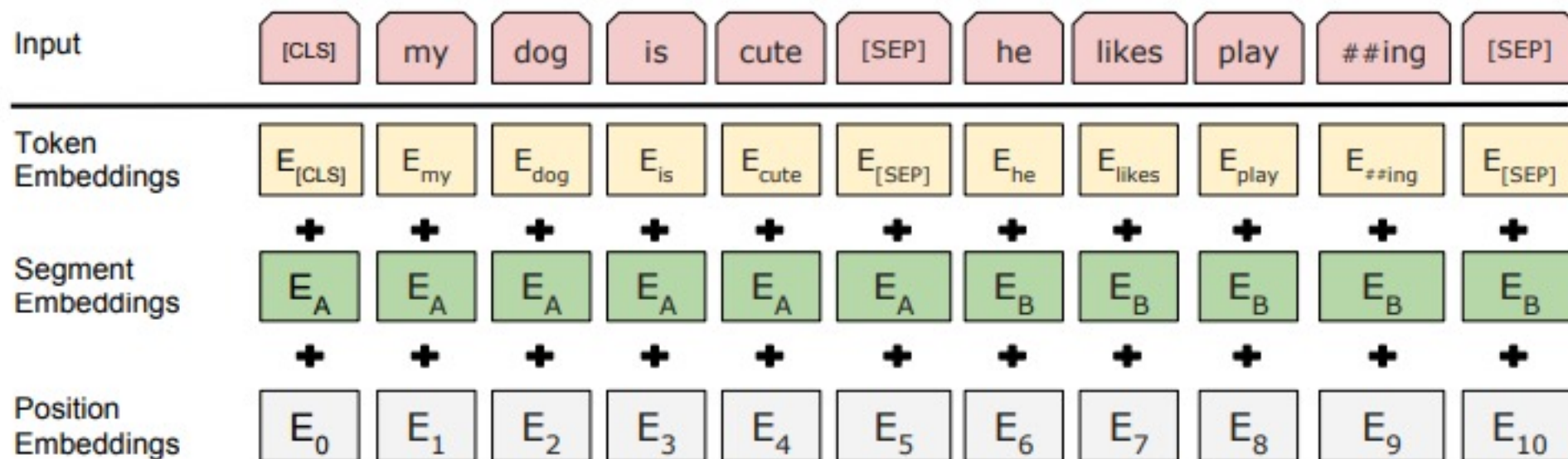


Figure 2: BERT input representation. The input embeddings are the sum of the token embeddings, the segmentation embeddings and the position embeddings.

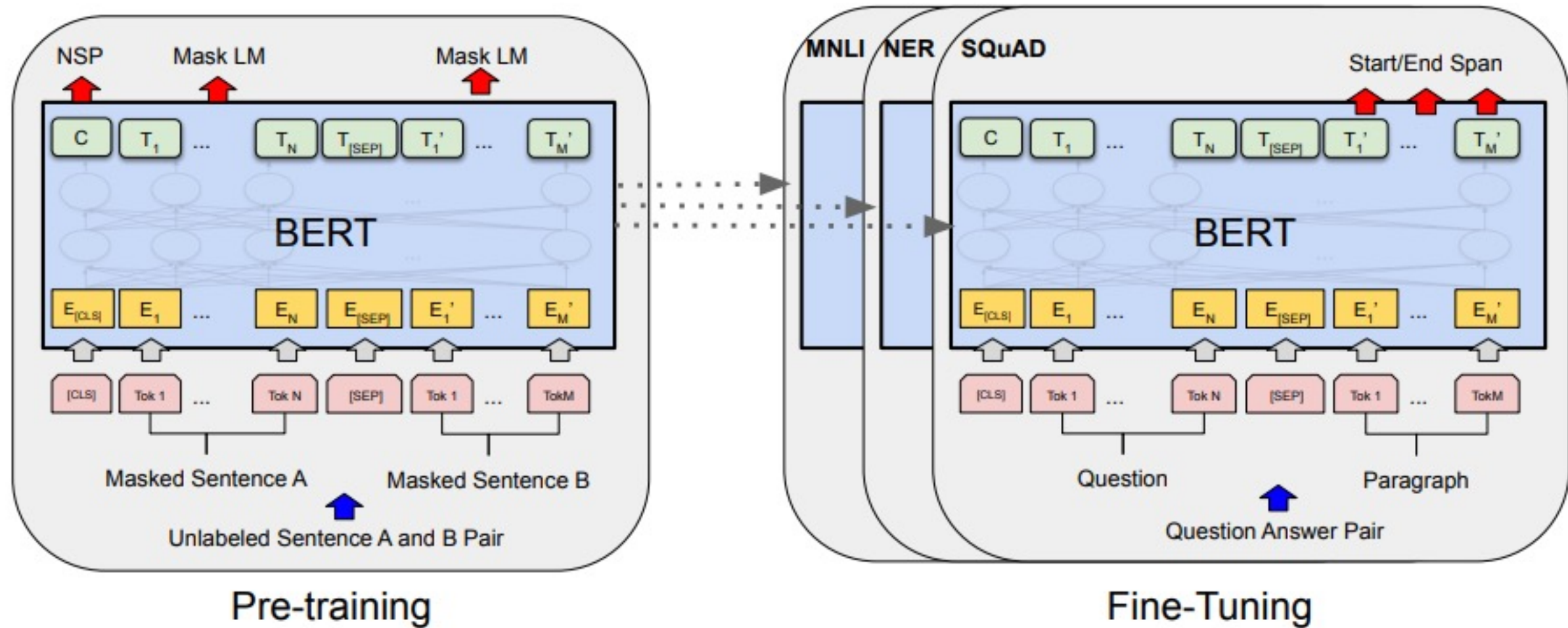
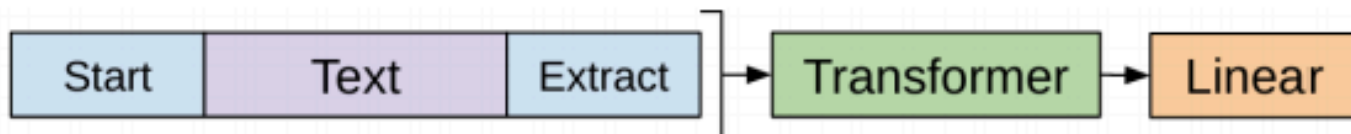
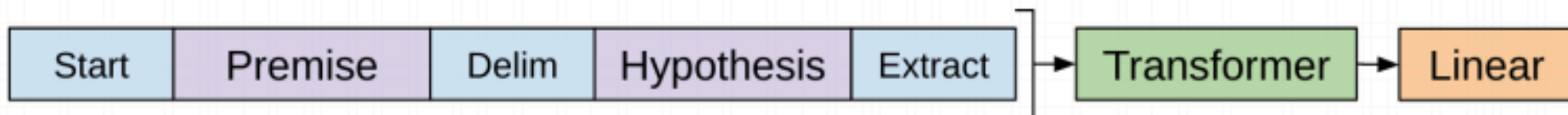


Figure 1: Overall pre-training and fine-tuning procedures for BERT. Apart from output layers, the same architectures are used in both pre-training and fine-tuning. The same pre-trained model parameters are used to initialize models for different down-stream tasks. During fine-tuning, all parameters are fine-tuned. [CLS] is a special symbol added in front of every input example, and [SEP] is a special separator token (e.g. separating questions/answers).

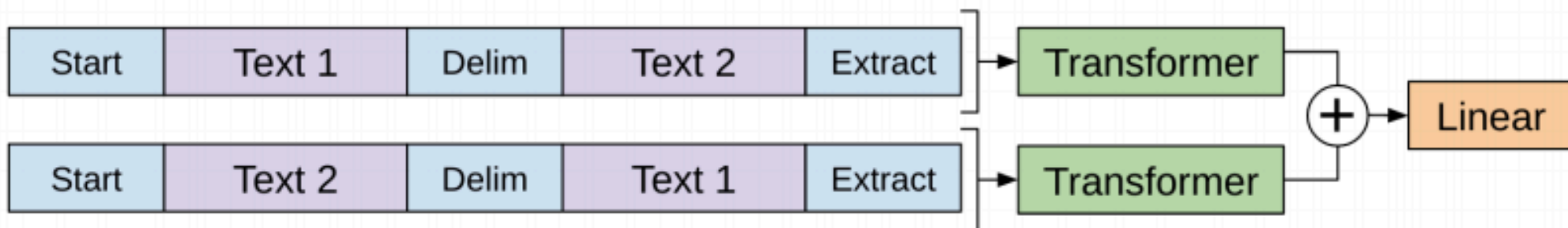
Classification



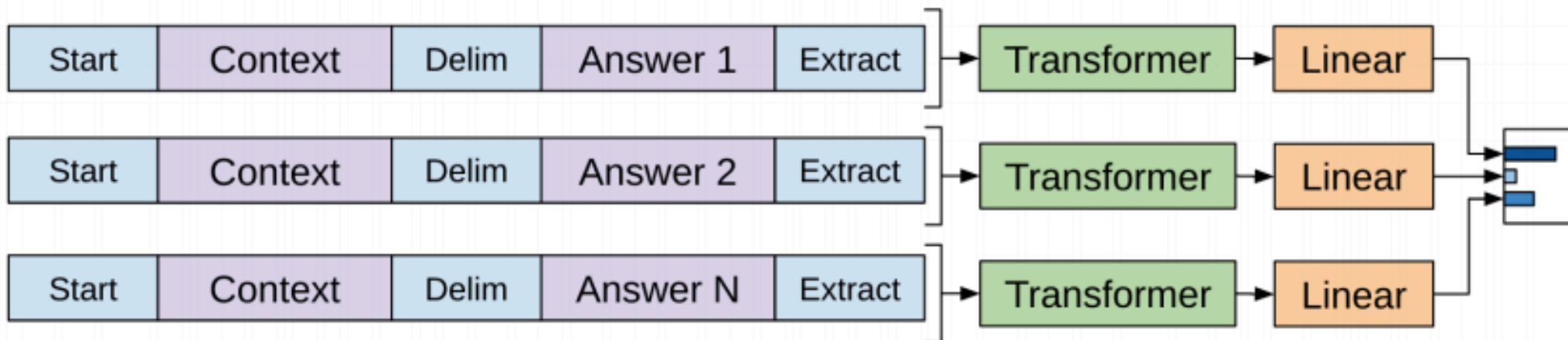
Entailment



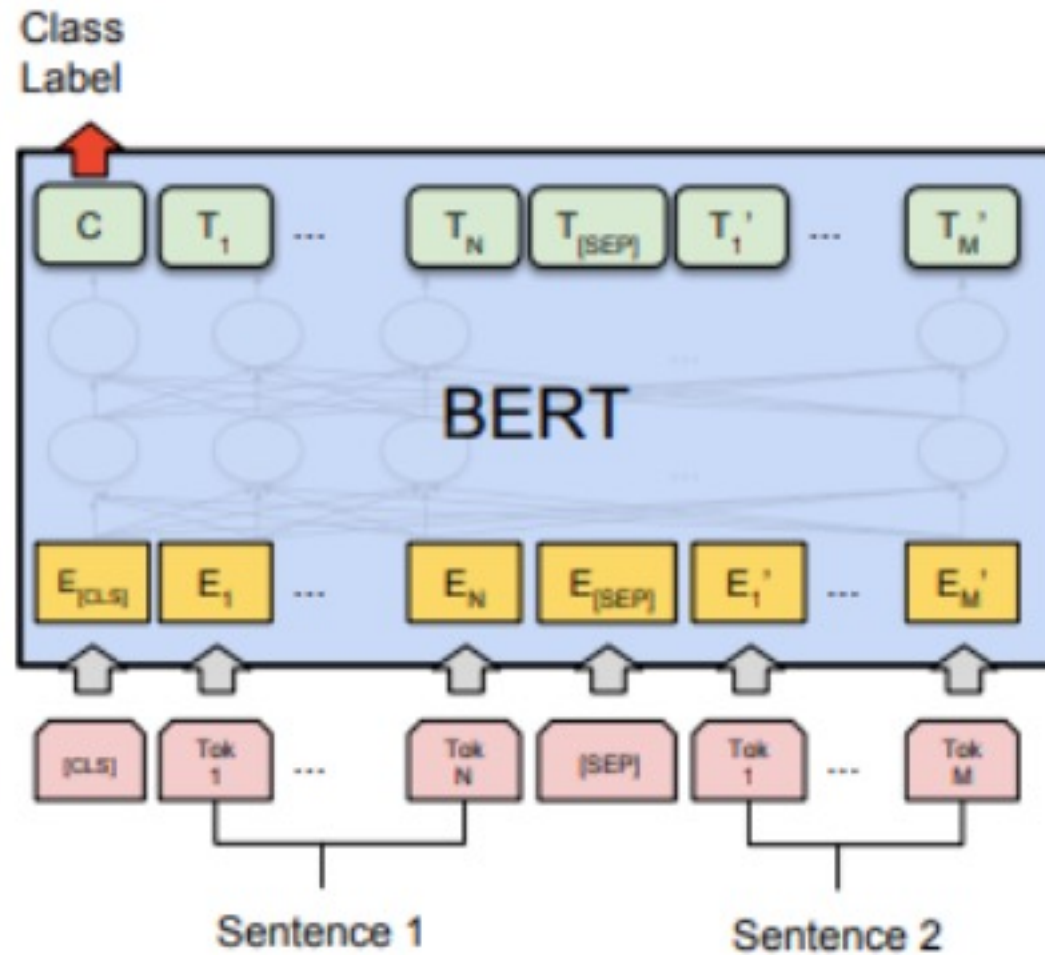
Similarity



Multiple Choice

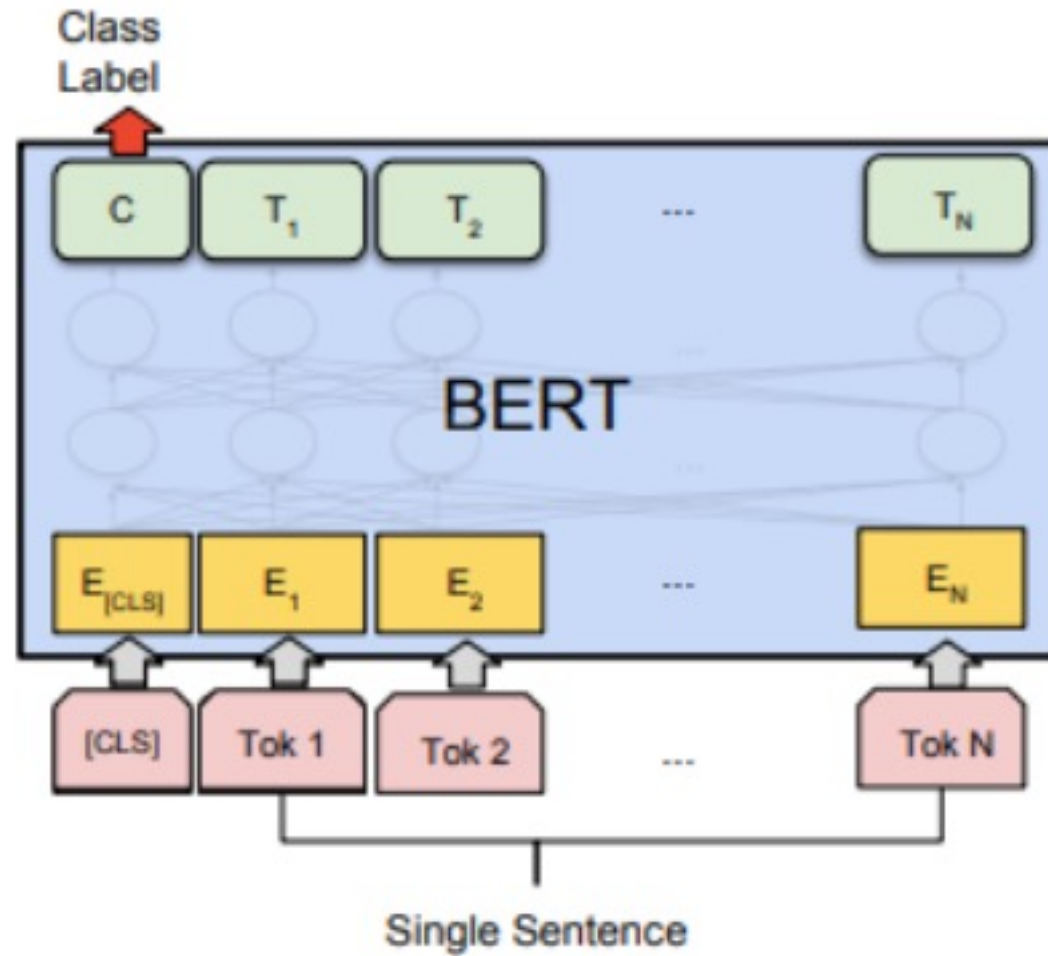


BERT fine-tuning



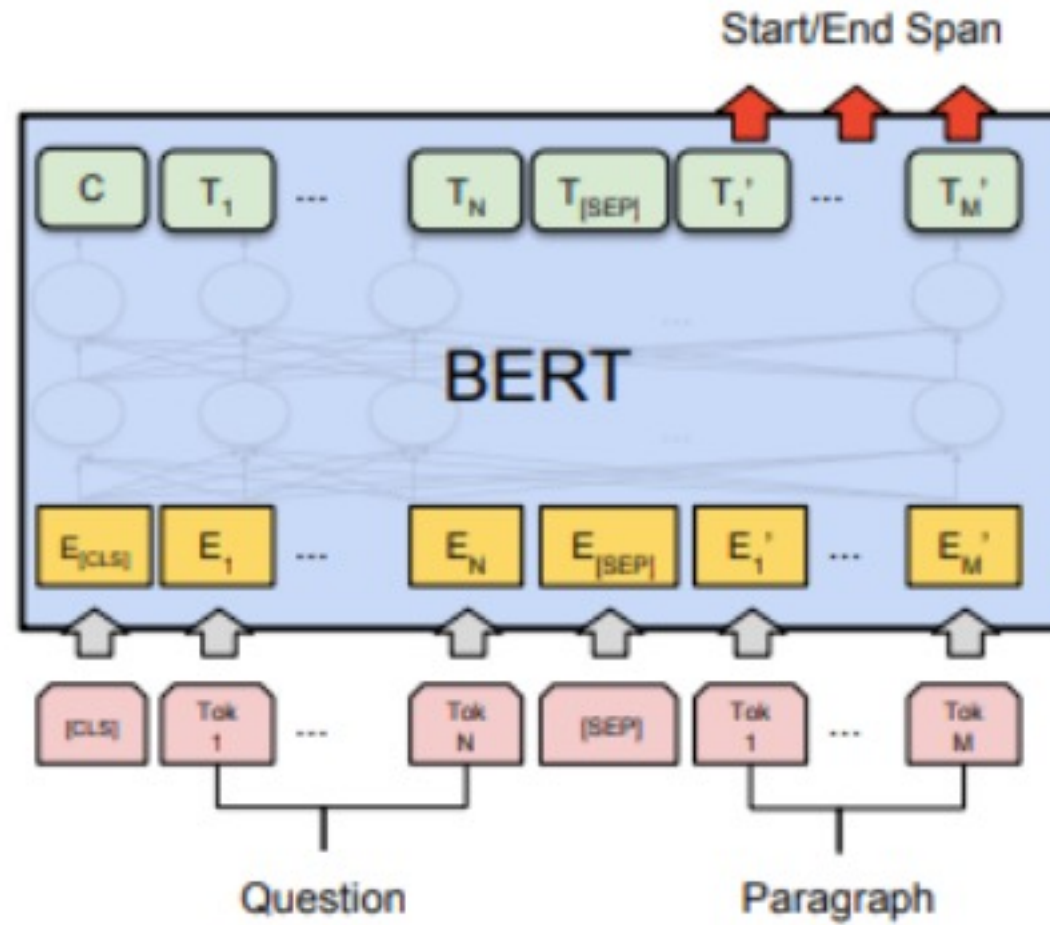
(a) Sentence Pair Classification Tasks:
MNLI, QQP, QNLI, STS-B, MRPC,
RTE, SWAG

BERT fine-tuning



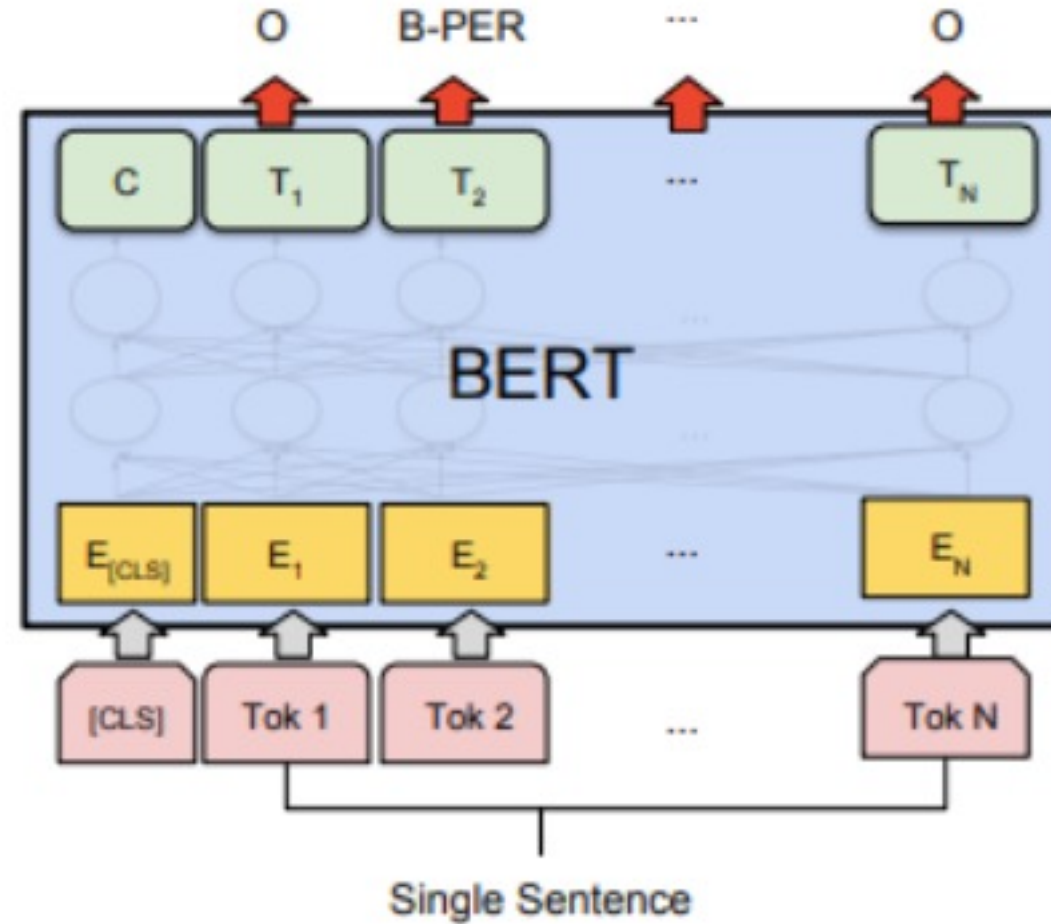
(b) Single Sentence Classification Tasks:
SST-2, CoLA

BERT fine-tuning



(c) Question Answering Tasks:
SQuAD v1.1

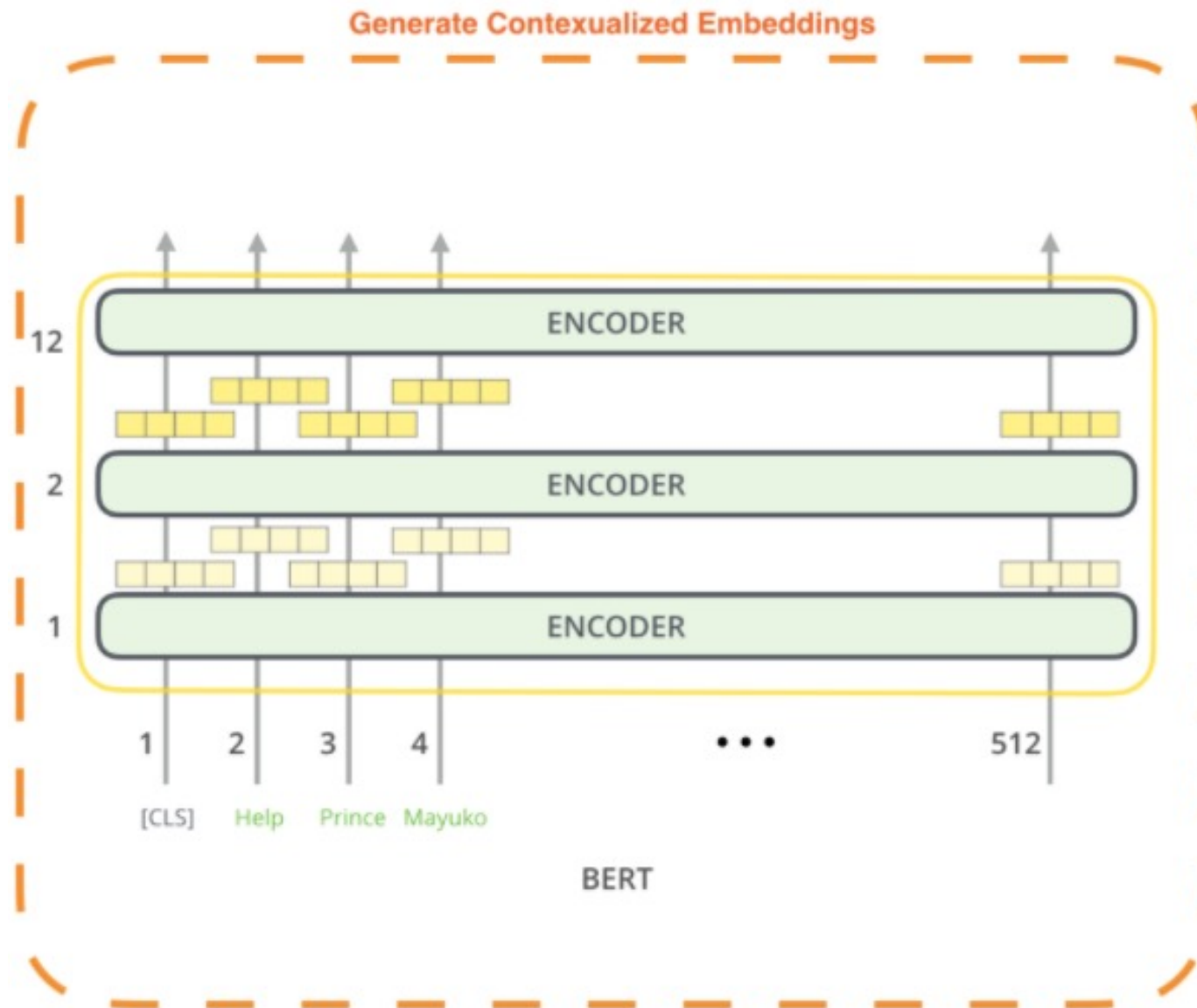
BERT fine-tuning



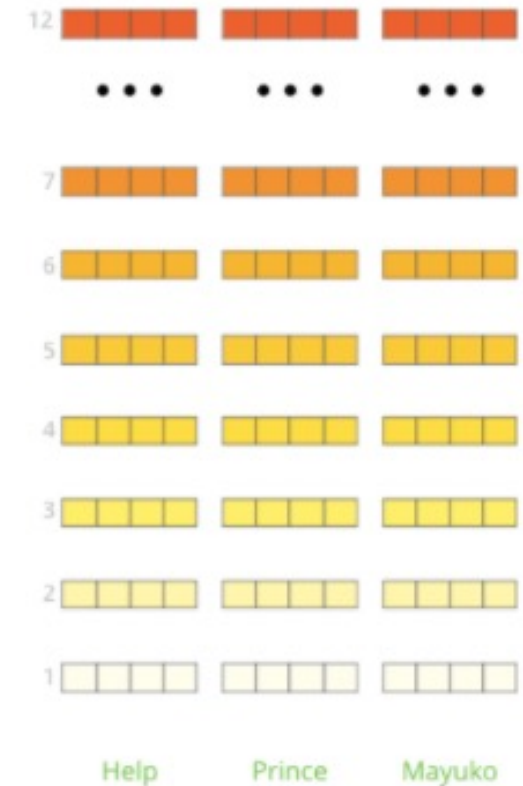
(d) Single Sentence Tagging Tasks:
CoNLL-2003 NER

BERT

Or, one could extract the **contextualized embeddings**



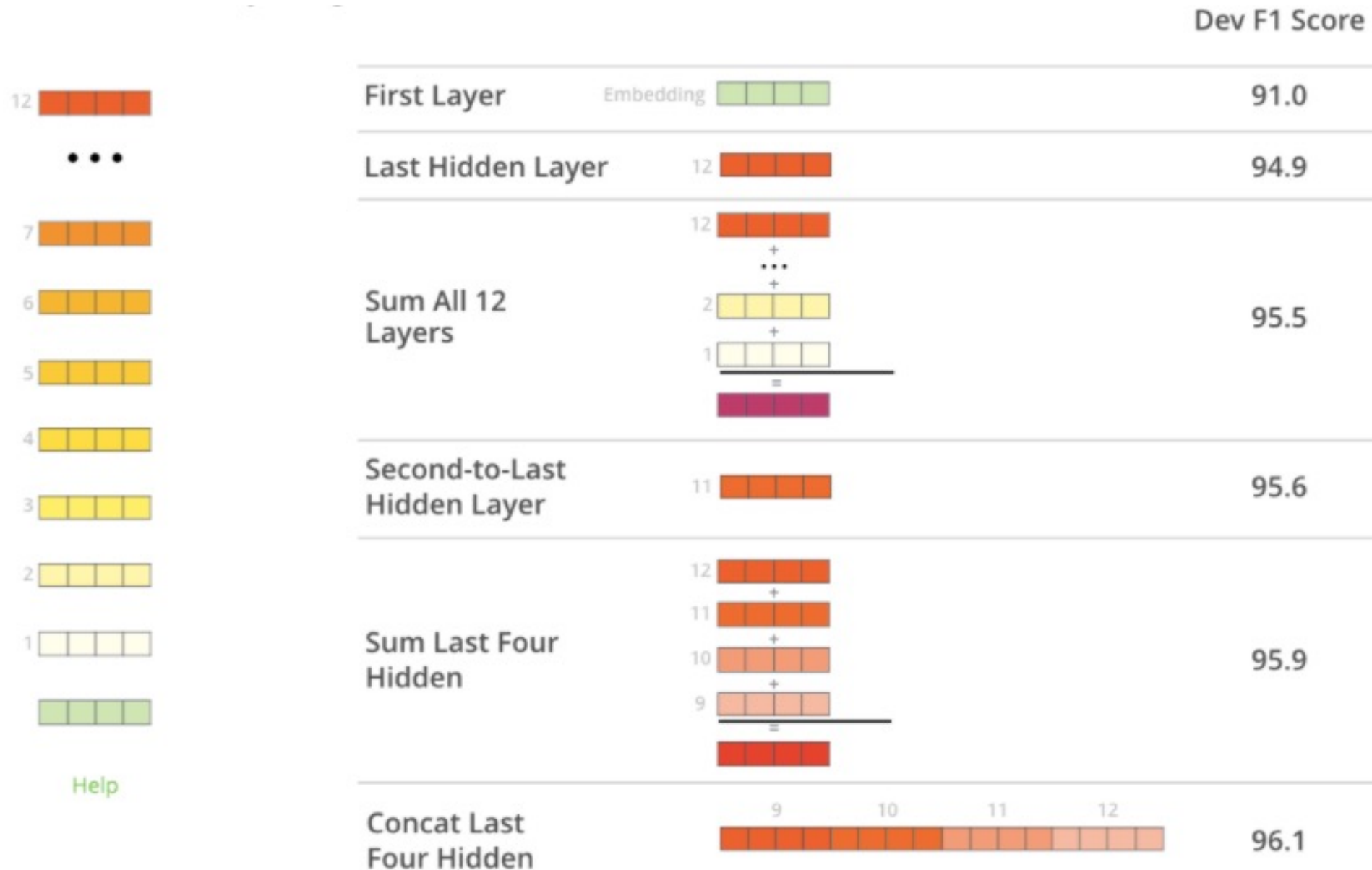
The output of each encoder layer along each token's path can be used as a feature representing that token.



But which one should we use?

BERT

Later layers have the best contextualized embeddings



BERT

BERT yields state-of-the-art (SOTA) results on many tasks

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	Average -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
BERT _{BASE}	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
BERT _{LARGE}	86.7/85.9	72.1	92.7	94.9	60.5	86.5	89.3	70.1	82.1

Table 1: GLUE Test results, scored by the evaluation server (<https://gluebenchmark.com/leaderboard>).

Takeaway

BERT is incredible for learning contextualized embeddings of words and using transfer learning for other tasks (e.g., classification).

Can't generate new sentences though, due to no decoders.

